

CPPDescent: A C++ library for the creation of K-NN graphs from multi-dimensional datasets

Software Development for Computing Systems, Winter 2023-2024

Department of Informatics and Telecommunications, University of Athens

Konstantinos Chousos*

Anastasios-Phaedon Seitanidis†

January 18, 2024

Abstract

This library is a C++ implementation of the “NN-Descent” algorithm by Dong et al. [4], with a few improvements and optimisations (e.g. usage of random projection trees [3]). This implementation serves as an entry for the [ACM SIGMOD 2023 competition](#). It was developed as part of the *Software Development for Computing Systems* course of the Department of Informatics and Telecommunications, taught in the winter of 2023 [9]. The source code is available at <https://github.com/kchousos/CPPDescent>.

1 Introduction

As is tradition for this course, our project for the semester was the challenge of last spring’s ACM SIGMOD Student Research Competition. The theme of the competition was the efficient and fast creation of a *K Nearest Neighbor* graph, where its vertices belong to a dataset of 100-dimensions vectors/datapoints of floats.

A brute force approach to this problem has a complexity of $O(n^2)$, since each datapoint needs to check all others so that it can *weed out* the K points nearest to it. An answer to this problem was given in 2011 by Dong et al. in their paper titled “Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures” [4], where they present the *NN-Descent* algorithm. This algorithm is based at the very simple idea that “*a neighbor of a neighbor is also likely to be a neighbor*”. According to them, this algorithm is shown to have a complexity of $O(n^{1.14})$. This is a major step-up from the quadratic time, especially

when it comes to big dataset.

Our library does not only implement the NN-Descent algorithm. It also implements other optimizations that e.g. make the starting graph more like the one we want to end up with, lowering this way the number of iterations the algorithm must do to reach a satisfying result. These optimizations are discussed in sec. 2.

1.1 Project layout

- The library itself is stored in the `src/` directory. It is compiled to a `.a` file, so that it can be linked with any executable.
- The main app that results to the main executable is in the `app/` directory. This file contains the only `main()` on the project, and can be thought as the API between the user and the library itself.
- The directory `datasets/` contains a lot of the datasets of the SIGMOD competition in binary format. The subdirectory `computed/` contains

*ID: 1115202000215

†ID: 1115202000179

some of the K-NN graphs that were computed using the brute force algorithm for comparison purposes. They are also in binary format. For which dataset and for which K they correspond to can be deduced from the respective filename.

- The docs/ subdirectory contains the html/ and latex/ subdirectories that they themselves contain files respective to each format. It is documentation of the code, automatically generated by code comments using [Doxygen](#). The lcov/ subdirectory contains files that are generated by [LCOV](#) and are used to monitor the code coverage provided by the tests (see below)¹,
- The extern/ directory contains the source code for the [Google Test](#) suite. It is included as a git submodule.
- The include/ directory contains header files for the library and the home-made data structures it uses.
- The test/ directory contains unit tests for the library.
- The helper.sh file is a bash script that is used to automate some repetitive tasks, such as execution of all the tests, or documentation generation.
- The experiments.sh is a script that executes the main program in quiet mode, and outputs a .csv file that contains the results of different executions for different parameters (see sec. 3).

1.2 Data structures

2 Optimizations

3 Experiments

3.1 Experimental Setup

3.1.1 Performance Measures

3.1.2 Default Parameters

3.1.3 System Environment

3.2 Experimental Results

3.2.1 Performance

3.2.1.1 Execution Speed

3.2.1.2 Memory Consumption

3.2.1.3 Single vs. Multi-Threading

3.2.2 Performance as Data Scales

3.2.3 Tweaking the Parameters

4 Conclusion

5 Acknowledgements

References

- [1] J. Brugger, *Brj0/nndescent*, Dec. 11, 2023. [Online]. Available: <https://github.com/brj0/nndescent>.
- [2] K. Chousos, *Kchousos/CPPDescent*, Jan. 18, 2024. [Online]. Available: <https://github.com/kchousos/CPPDescent>.
- [3] S. Dasgupta and Y. Freund, “Random projection trees and low dimensional manifolds,” in *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, Victoria British Columbia Canada: ACM, May 17, 2008, pp. 537–546, ISBN: 978-1-60558-047-0. DOI: [10.1145/1374376.1374452](https://doi.org/10.1145/1374376.1374452). [Online]. Available: <https://dl.acm.org/doi/10.1145/1374376.1374452>.

¹At the moment, the code coverage of the cppdescent library itself is pretty low, since it went very recently under a large refactoring and overhaul.

- [4] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in *Proceedings of the 20th International Conference on World Wide Web*, ser. WWW ’11, New York, NY, USA: Association for Computing Machinery, Mar. 28, 2011, pp. 577–586, ISBN: 978-1-4503-0632-4. doi: [10.1145/1963485.1963487](https://doi.org/10.1145/1963485.1963487). [Online]. Available: <https://dl.acm.org/doi/10.1145/1963485.1963487>.
- [5] Enthought, director, *PyNNDescent Fast Approximate Nearest Neighbor Search with Numba | SciPy 2021*, Jul. 23, 2021. [Online]. Available: https://www.youtube.com/watch?v=xPadY4_kt3o.
- [6] *Google/googletest*, Google, Jan. 18, 2024. [Online]. Available: <https://github.com/google/googletest>.
- [7] “GSL - GNU Scientific Library - GNU Project - Free Software Foundation.” (), [Online]. Available: <https://www.gnu.org/software/gsl/>.
- [8] “How PyNNDescent works — pynndescent 0.5.0 documentation.” (), [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html.
- [9] Γ. Ιωαννίδης. “Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα.” (), [Online]. Available: <https://eclass.uoa.gr/courses/DI485/>.
- [10] D. Kluser, J. Bokstaller, S. Rutz, and T. Buner. “Fast Single-Core K-Nearest Neighbor Graph Computation.” arXiv: [2112.06630](https://arxiv.org/abs/2112.06630) [cs, math]. (Dec. 13, 2021), [Online]. Available: <http://arxiv.org/abs/2112.06630>, preprint.
- [11] OpenMP Architecture Review Board, *OpenMP application program interface version 5.2*, Nov. 2021. [Online]. Available: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>.
- [12] Stitch Fix Multithreaded, director, *Algo Hour - Nearest Neighbor Descent (and friends) | Dr. Leland McInnes*, May 22, 2020. [Online]. Available: https://www.youtube.com/watch?v=0vT2NY_FV_g.